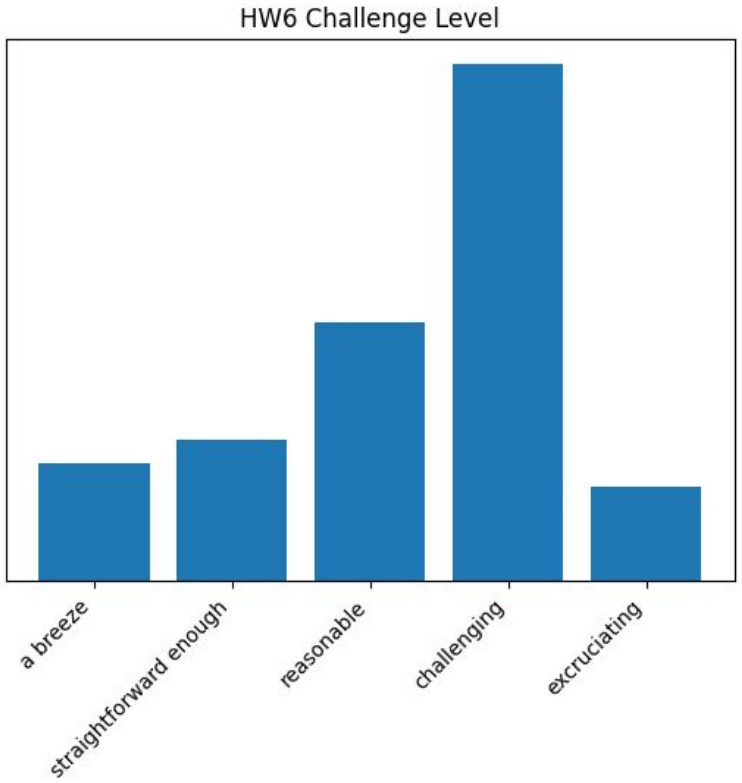
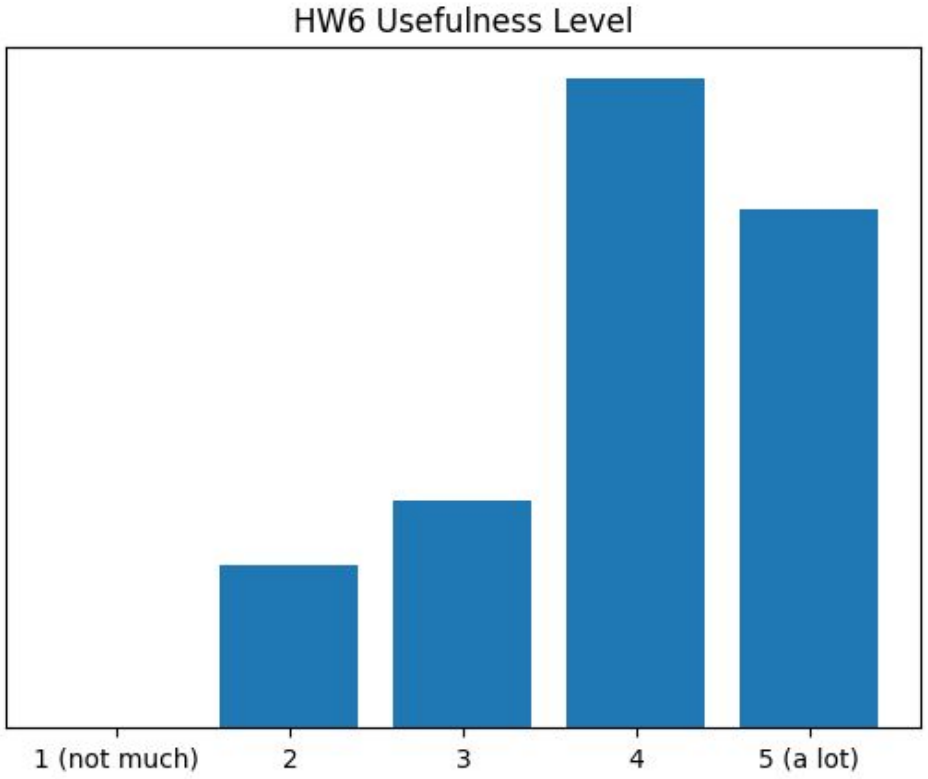
CSE 493L Section 8

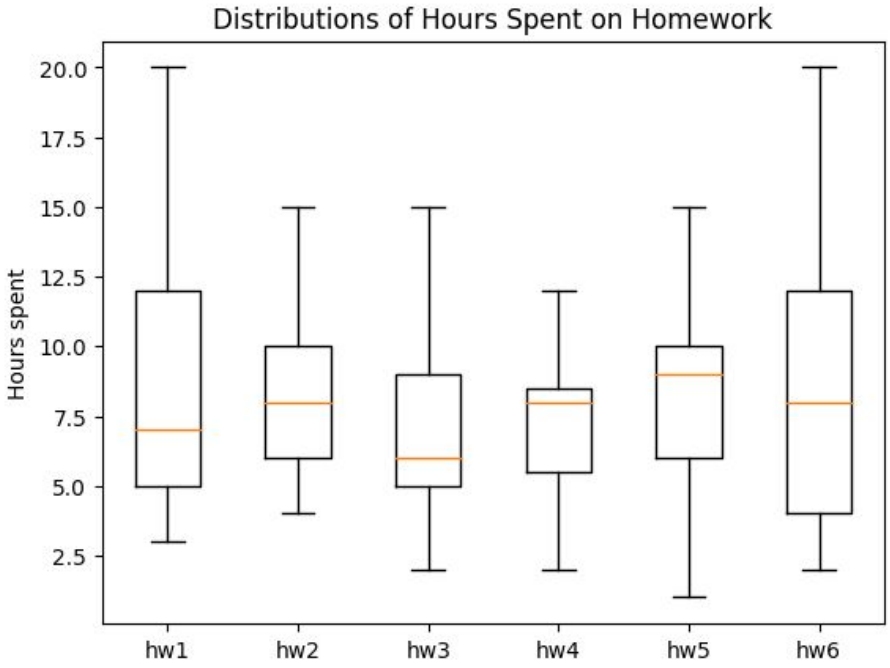
Threadpool implementation
(and a bit of async)

Looking Back on Homework 6





Time Spent Across Homeworks



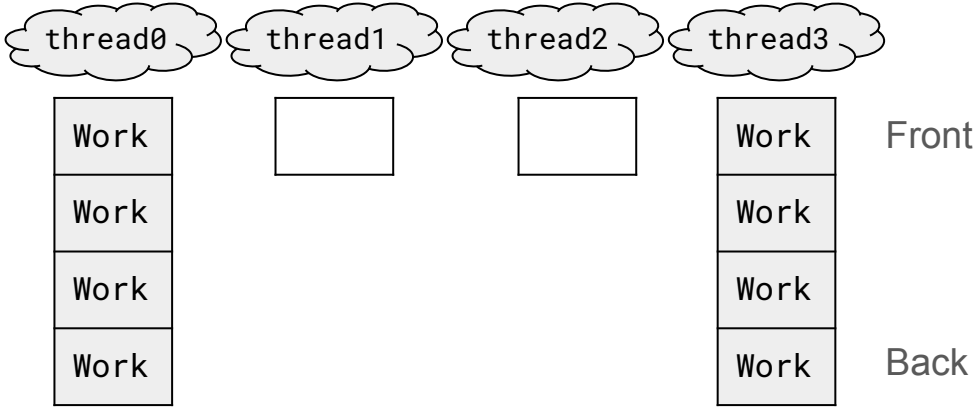
Notes

- The homeworks are easier for students that ask questions in office hours.
  - Let us know if you're lost – we'll re-explain any concept step-by-step.
- Don't stress: students aren't expected to get 100% on all homeworks.
- This class is hard and all of you are doing very well! Great job!

ThreadPool

ThreadPool

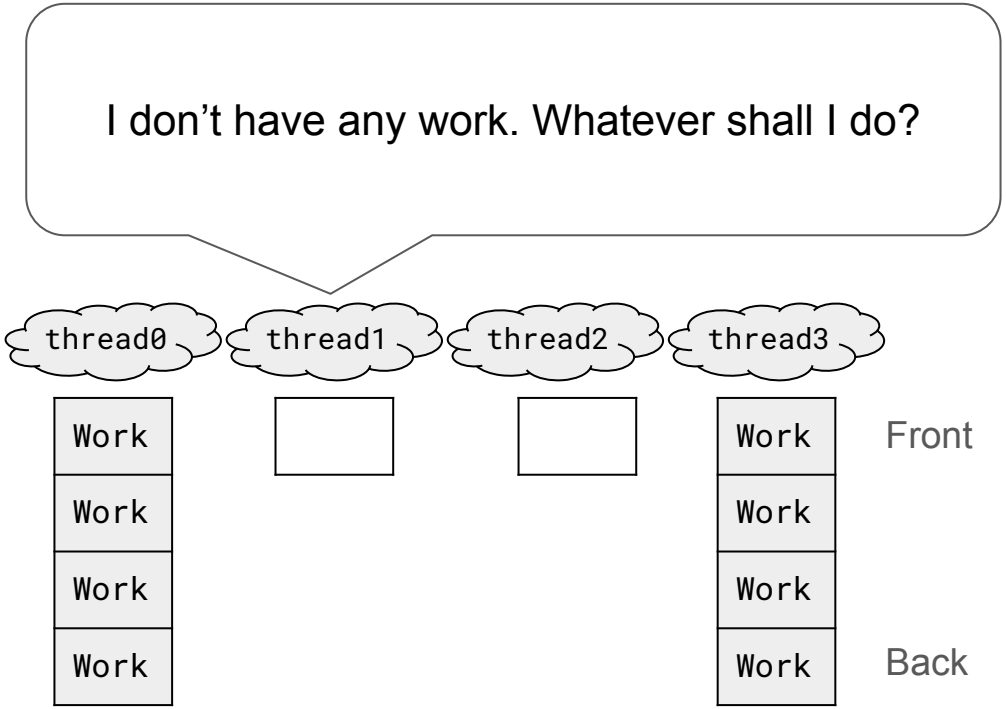
- Each thread pulls work from the front of its own queue, to minimize contention.
- Each piece of work is a closure that may submit more pieces of work.
  - Such work should be pushed to the front of the current thread's queue.



Work Stealing

`StealMode`

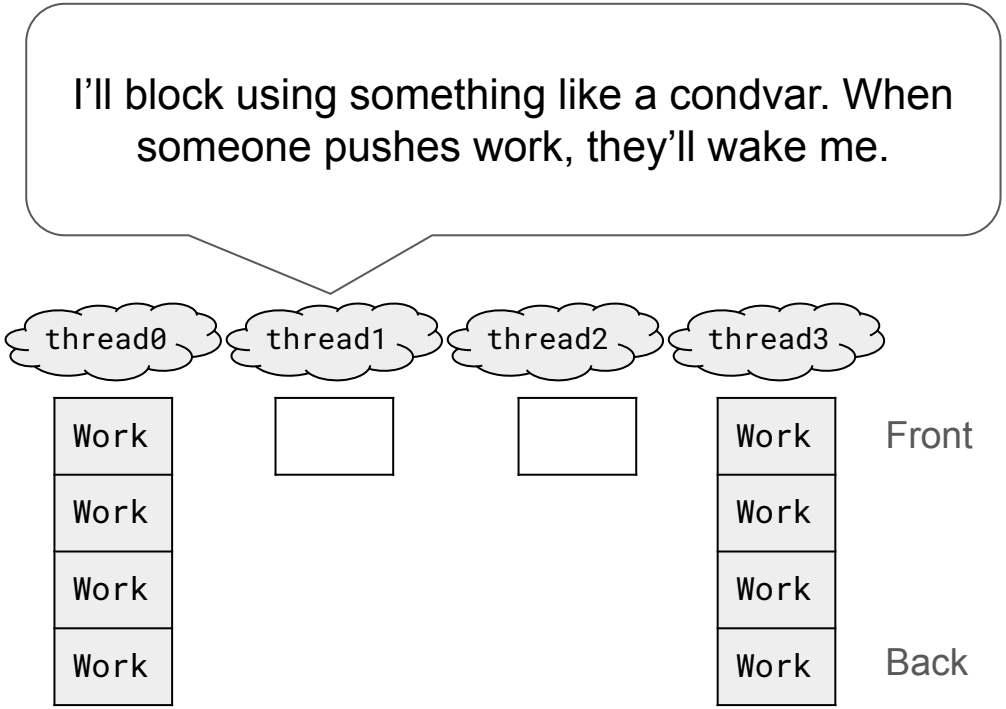
- `None`
- `OneAtATime`
- `TakeHalf`



Work Stealing

`StealMode`

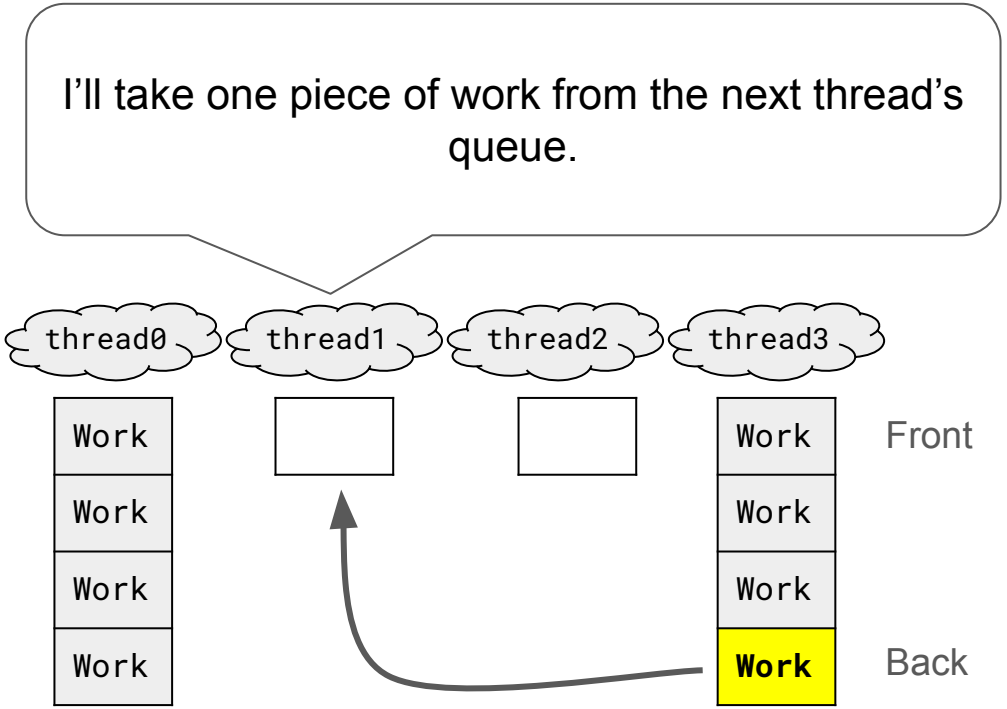
- **None**
- `OneAtATime`
- `TakeHalf`



Work Stealing

`StealMode`

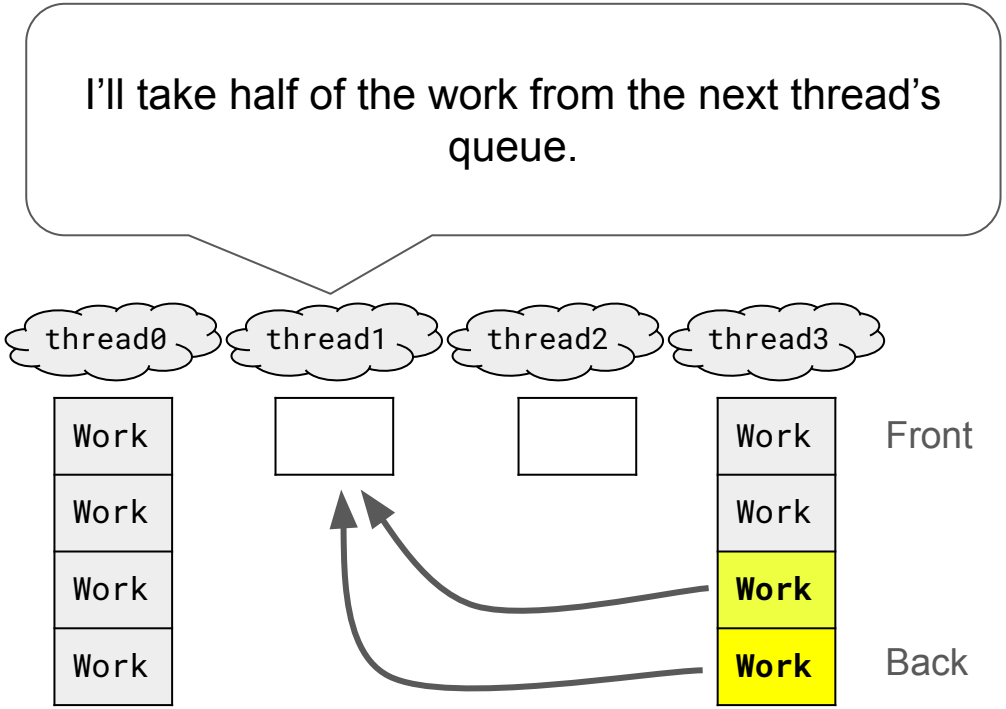
- `None`
- **`OneAtATime`**
- `TakeHalf`



Work Stealing

`StealMode`

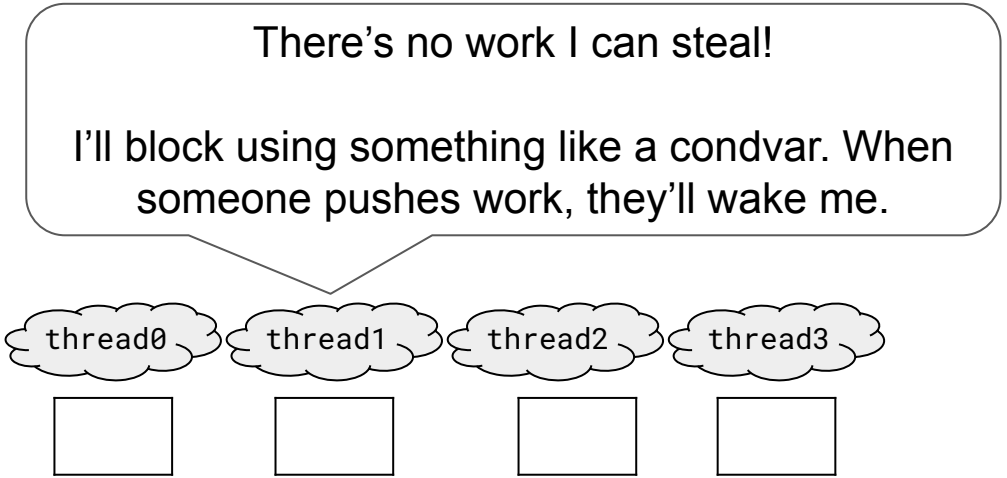
- `None`
- `OneAtATime`
- **`TakeHalf`**



Work Stealing

`StealMode`

- `None`
- `OneAtATime`
- `TakeHalf`

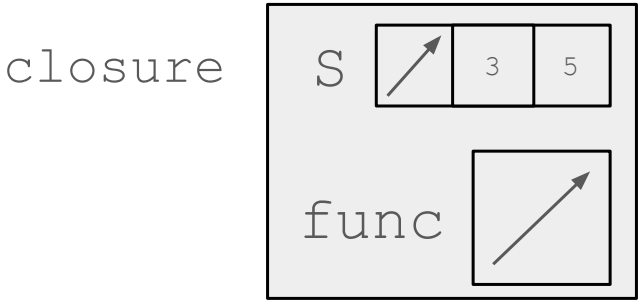


Closures

The move keyword moves all needed environment variables into the closure.

```
let s = String::from("hi");
let closure = move || {
    println!("{s}");
};

closure();
```

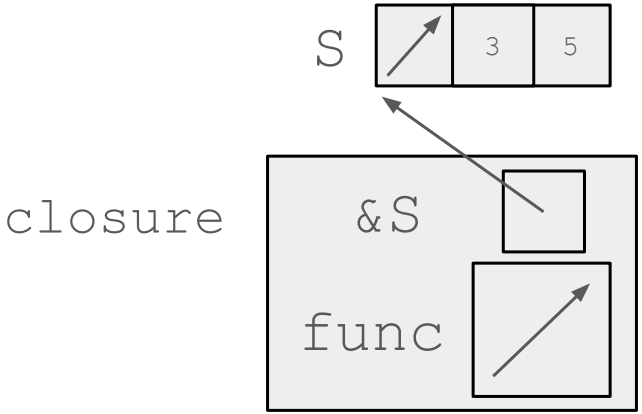


Closures

The move keyword moves all needed environment variables into the closure.

```rust
let s = String::from("hi");
let closure = || {
    println!("{s}");
};

println!("{s}");
closure();
```

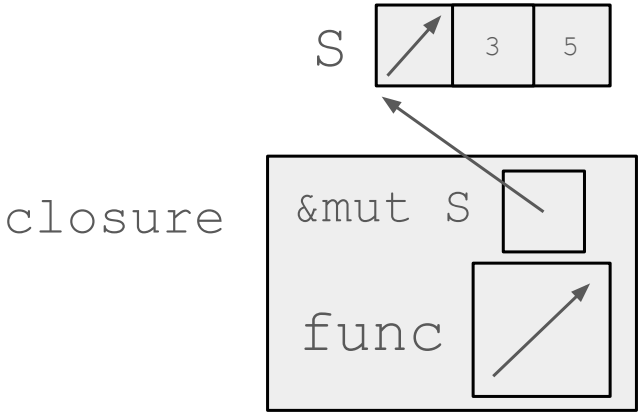


Closures

The move keyword moves all needed environment variables into the closure.

```
let mut s = String::from("hi");
let closure = || {
    s.push('!');
    println!("{s}");
};

println!("{s}");
closure();
```



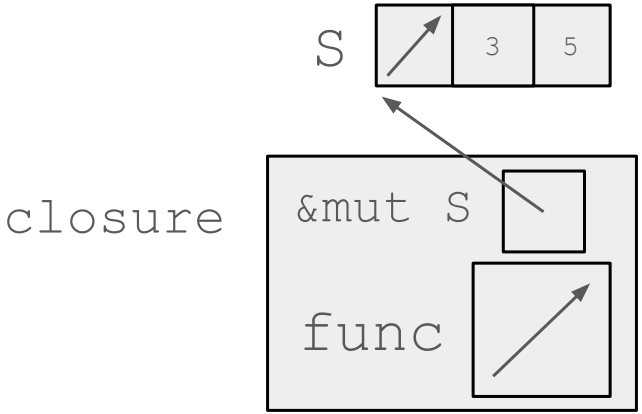
Will this compile?

Closures

The move keyword moves all needed environment variables into the closure.

```
let mut s = String::from("hi");
let closure = || {
    s.push('!');
    println!("{s}");
};

println!("{s}");
closure();
```



Will this compile? NO! Closure continues to hold `&mut s`, cannot borrow `s` again.

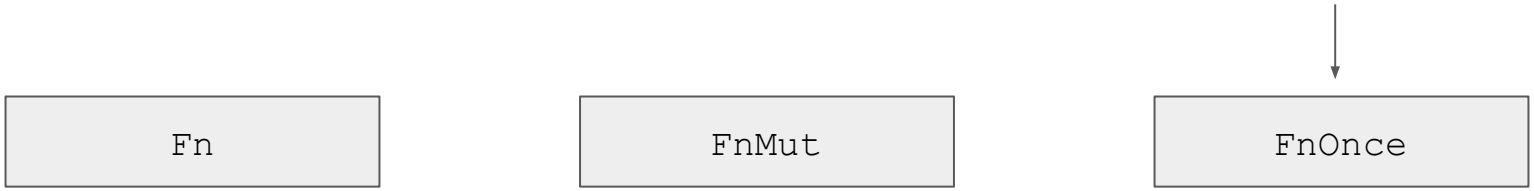
** maybe something is moved out of the closure after it is called*

The Closure Hierarchy

Rust closures *always have unique types*

- Accounts for differences in environment, and allows for better optimization
- Each closure implements one of three traits based off what it does

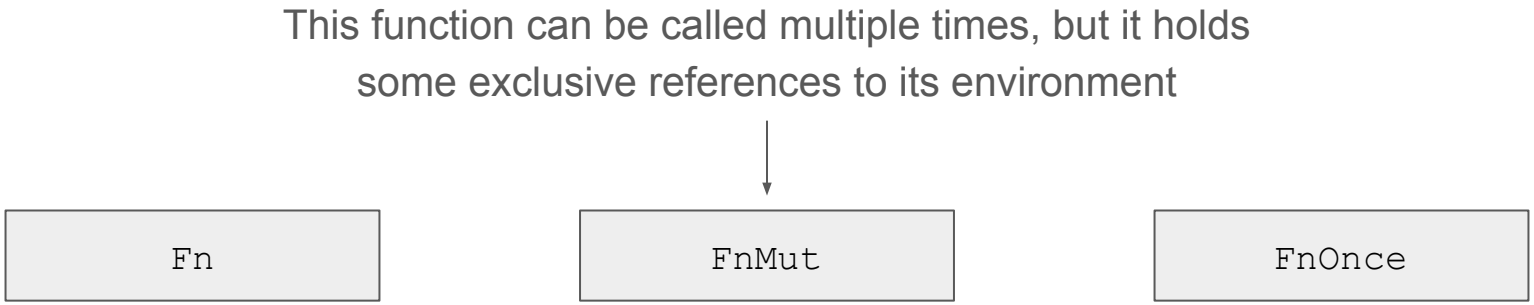
This function can be called once, as it cannot assume it's environment will stay intact after*



The Closure Hierarchy

Rust closures *always have unique types*

- Accounts for differences in environment, and allows for better optimization
- Each closure implements one of three traits based off what it does

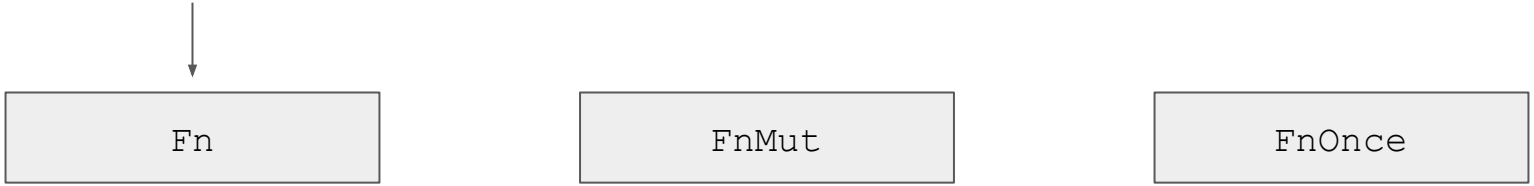


The Closure Hierarchy

Rust closures *always have unique types*

- Accounts for differences in environment, and allows for better optimization
- Each closure implements one of three traits based off what it does

This function can be called multiple times, and holds no exclusive references within its environment



The Closure Hierarchy

Rust closures *always have unique types*

- Accounts for differences in environment, and allows for better optimization
- Each closure implements one of three traits based off what it does

Is a function which is `FnMut`, also `FnOnce`?

`Fn` `FnMut` `FnOnce`

The Closure Hierarchy

Rust closures *always have unique types*

- Accounts for differences in environment, and allows for better optimization
- Each closure implements one of three traits based off what it does

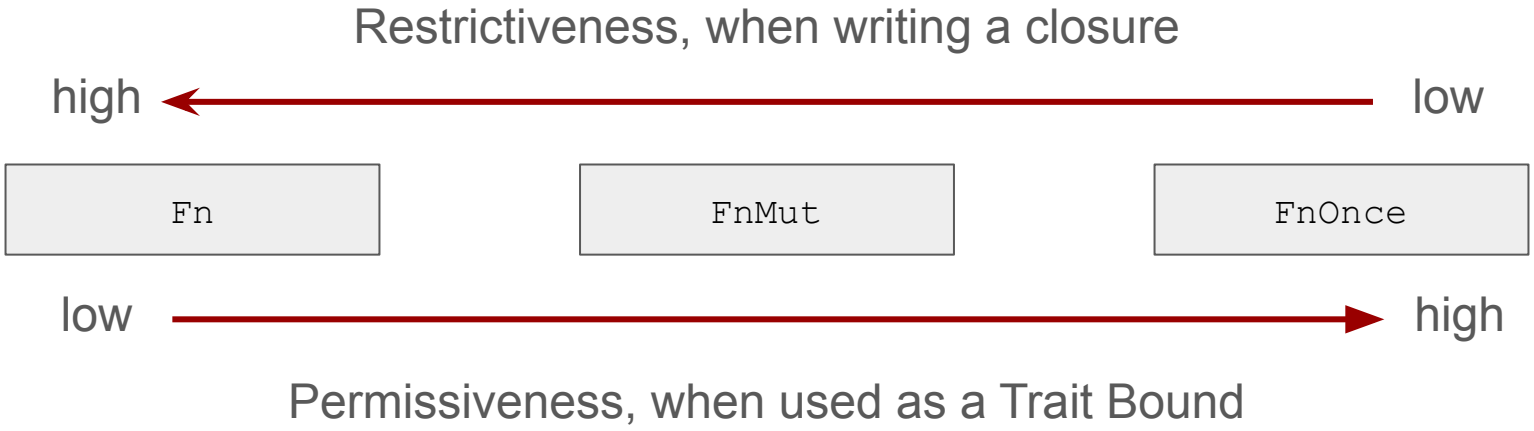
Is a function which is `FnMut`, also `Fn`?

`Fn` `FnMut` `FnOnce`

The Closure Hierarchy

Rust closures *always have unique types*

- Accounts for differences in environment, and allows for better optimization
- Each closure implements one of three traits based off what it does



Closures

Closures capture variables in the scope of that they are defined in

What’s the size of this closure?

```
...
let closure = || {
    ...
};
...
```

Closures

Closures capture variables in the scope of that they are defined in

What's the size of this closure?

It depends on the captured environment

```
...
let closure = || {
    ...
};
...
```

We want to create queues of closures!! How can we do that if it's `!Sized`?

Closures

Closures capture variables in the scope of that they are defined in

What's the size of this closure?

It depends on the captured environment

```
...
let closure = || {
    ...
};
...
```

We want to create queues of closures!! How can we do that if it's `!Sized`?

- Introduce indirection to a trait object, e.g. `Box<dyn FnOnce()>`.
- `Vec<Box<dyn FnOnce()>>` can then be constructed as `Box` is sized!

Creating Work via Closures

We expect we are going to need to move these closures between threads…

Can a `Box<dyn FnOnce()>` be moved between threads?

```
let s = Rc::new(String::from("hi"));
let closure = move || {
   println!("{s:?}");
};
```

Creating Work via Closures

We expect we are going to need to move these closures between threads…

Can a `Box<dyn FnOnce()>` be moved between threads?

- No, it may capture a type which is NOT `Send`!
- No, it may capture a reference, which does not live long enough!

```
let s = Rc::new(String::from("hi"));
let closure = move || {
   println!("{s:?}");
};
```

Creating Work via Closures

We expect we are going to need to move these closures between threads…

Can a `Box<dyn FnOnce()>` be moved between threads?

- No, it may capture a type which is NOT `Send`!
- No, it may capture a reference, which does not live long enough!

```
let s = Rc::new(String::from("hi"));
let closure = move || {
   println!("{s:?}");
};
```

We can use `type Work = Box<dyn FnOnce() + Send + 'static>`!

The ThreadPool API

```
impl ThreadPool {
   pub fn new(num_threads: usize, steal_mode: StealMode) -> Self {
       todo!()
   }
}
```

Returns a new `ThreadPool` struct

- Spawns worker threads
- Creates queues, properly shares them between each thread
- Creates extra state required to wake threads that had no work previously
- Creates extra state required to manage stopping the pool

The ThreadPool API

```rust
impl ThreadPool {
    pub fn new(num_threads: usize, steal_mode: StealMode) -> Self {
        todo!()
    }
}
```

What should each thread run when it’s created?

The ThreadPool API

```rust
impl ThreadPool {
    pub fn new(num_threads: usize, steal_mode: StealMode) -> Self {
        todo!()
    }
}
```

What should each thread run when it's created?

- Run some function (`thread_start`) which repeatedly tries to pull from the thread's queue!
  - If the thread's queue is empty, try to steal based off the `steal_mode`!
  - If the thread truly has nothing to run, block on a CV, waiting for work to be submitted.
  - If the ThreadPool has been stopped, then stop polling the queue!

The ThreadPool API

```rust
pub fn submit<F, RF>(&mut self, func: F) -> JobHandle<RF>
where
    F: FnOnce() -> RF + Send + 'static,
    RF: Send + 'static,
{
    todo!()
}
```

Submits a piece of work to the ThreadPool.

- `JobHandle<RF>` can be used to wait for the return value of type `RF`.
- If called outside the threadpool – pick queue in cyclic order, and push to front.
- If called on a worker thread – push to the front of that thread's queue.

Creating `Work` Out of `func`s

Hold on! The submitted `func` doesn't have the same type as `Work`!

We store the following in each thread's queue:

```
type Work = Box<dyn FnOnce() + Send + 'static>;
```

But we submit closures to our ThreadPool with signature:

```
F: FnOnce() -> RF + Send + 'static
```

How can we convert a `FnOnce() -> RF` to a `FnOnce()`?

Creating `Work` Out of `funcs`

How can we convert a `FnOnce() -> RF` to a `FnOnce()`?

```rust
let f = || -> usize {
    // do stuff!
    // ...
    100
};
```

```rust
fn submit<F, RF>(mut f: F)
where
    F: FnOnce() -> RF + Send + 'static,
{
    let mut work_res = None;
    let work = || {
        let f_res = f();
        work_res.replace(f_res);
        // Notify the handle that I'm done!
    };

    // at some point, after work() is executed,
    // work_res is set to Some(RF)! It would be
    // nice if the JobHandle had it...
}
```

`f`'s return value is moved out of the work closure, and therefore can be accessed elsewhere!

The whole point here is that `Work` is not generic over `RF`, but "can operate as if it was"

The JobHandle API

```rust
impl<RF> JobHandle<RF> {
   pub fn get_result(self) -> Option<RF> {
       todo!()
   }
}
```

Gets the return value from running some previously submitted work (`RF`)

- If ThreadPool is stopped, returns **None**.
- If the associated Work is not yet finished running:
  - If called from outside the ThreadPool – block until return value is ready!
  - If called from a worker thread – do other work until the return value is ready.
    - Fun fact: this is *kind of like* a user-space context switch!

Use a `Condvar` based monitor!

The ThreadPool API

```
impl ThreadPool {
   pub fn stop(&mut self) {
       todo!()
   }
}
```

Tells all worker threads to stop doing any new work.

- Set a flag in each thread, when this thread looks for any new work, check this flag first!
  - Joins all worker threads after that!
- Causes `job_handle.get_result()` to return `None`.

Thread Local

`ThreadPool::submit()` and `JobHandle::get_result()` behave differently depending on whether they were called by a worker thread.

- Use thread local variables to maintain thread-specific state!

These vars are like global (aka `static`) vars, but ***each*** thread has its own copy.

- Destructed when the thread stops.
- At the least, you will need:
  - An `Arc` pointer to this thread's ThreadPool (maybe **None** if it's not a worker)
  - The worker thread's index (**None** if it's not a worker, i.e. a thread outside the pool)
  - … whatever other state you decide will be useful!

Thread Local

```rust
thread_local! {
   // BAR starts as None
   static BAR: RefCell<Option<String>> = RefCell::new(None); // starts as None
}
```

All code sections within each thread can potentially access `BAR`.

- Therefore, can't access `&mut BAR`. We can only access `&BAR`.
  - Use `RefCell` for interior mutability.
  - `RefCell` panics if there's more than one mutable references to inner `Option<String>`.
- Initialize all worker `thread_local!` state in `thread_start`

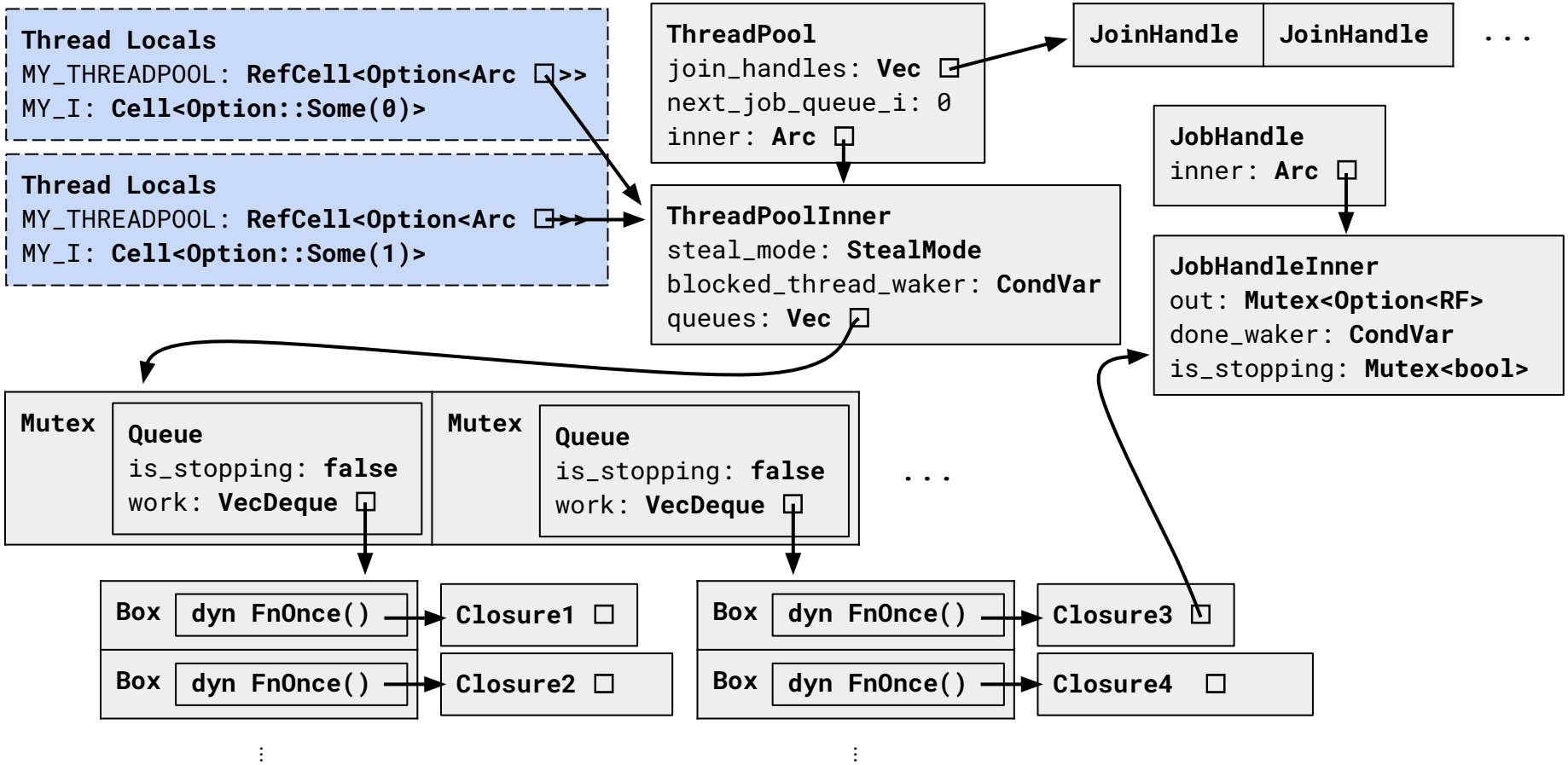
Thread Local

```rust
thread_local! {
   // BAR starts as None
   static BAR: RefCell<Option<String>> = RefCell::new(None); // starts as None
}

BAR.set(Some(String::from("hi"))); // initialize BAR
let s_clone = BAR.with_borrow(|inner: &Option<String>| inner.clone()); // access BAR
```

Can only access `&BAR` inside methods like `with_borrow()`.

- This is how Rust prevents `&BAR` from being given to another thread.
  - Otherwise, that `&BAR` would become dangling when `BAR` is destructed when its thread stops.
- You don’t need to understand this – it’s a quirk of Rust.


Thread Locals
MY_THREADPOOL: RefCell<Option<Arc >>
MY_I: Cell<Option::Some(0)>
Thread Locals
MY_THREADPOOL: RefCell<Option<Arc >>
MY_I: Cell<Option::Some(1)>
ThreadPool
join_handles: Vec
next_job_queue_i: 0
inner: Arc
JoinHandle
JoinHandle
...
JobHandle
inner: Arc
ThreadPoolInner
steal_mode: StealMode
blocked_thread_waker: CondVar
queues: Vec
JobHandleInner
out: Mutex<Option<RF>
done_waker: CondVar
is_stopping: Mutex<bool>
Mutex
Queue
is_stopping: false
work: VecDeque
Mutex
Queue
is_stopping: false
work: VecDeque
...
Box
dyn FnOnce()
Closure1
Box
dyn FnOnce()
Closure2
Box
dyn FnOnce()
Closure3
Box
dyn FnOnce()
Closure4

Async

`std::thread::spawn(|| {`
`  std_connection.read()`
`});`
	`tokio::task::spawn(async {`
	`  tokio_connection.read().await;`
	`})`

`read()` is a blocking system call.	**`read().await`** internally makes a non-blocking system call telling the OS to start an operation.

`std::thread::spawn(|| {`
`  std_connection.read()`
`});`
	`tokio::task::spawn(async {`
	`  tokio_connection.read().await;`
	`})`

`read()` is a blocking system call.	**`read().await`** internally makes a non-blocking system call telling the OS to start an operation.
Yields to the OS, so other spawned threads can run.	Yields to the userspace runtime (Tokio), so other spawned tasks can run on the same thread.

`std::thread::spawn(|| {`
`  std_connection.read()`
`});`
	`tokio::task::spawn(async {`
	`  tokio_connection.read().await;`
	`})`

`read()` is a blocking system call.	**`read().await`** internally makes a non-blocking system call telling the OS to start an operation.
Yields to the OS, so other spawned threads can run.	Yields to the userspace runtime (Tokio), so other spawned tasks can run on the same thread.
When blocking, the entire stack is saved.	When awaiting, only necessary variables are saved (in its “Future” state machine).

`std::thread::spawn(|| {`
`  std_connection.read()`
`});`
	`tokio::task::spawn(async {`
	`  tokio_connection.read().await;`
	`})`

`read()` is a blocking system call.	**`read().await`** internally makes a non-blocking system call telling the OS to start an operation.
Yields to the OS, so other spawned threads can run.	Yields to the userspace runtime (Tokio), so other spawned tasks can run on the same thread.
When blocking, the entire stack is saved.	When awaiting, only necessary variables are saved (in its “Future” state machine).
Threads are great for CPU-bound work like parallel prefix sums.	Async tasks are great for IO-bound work like waiting on a million users to send a request.

For each task → Use case ↓	thread::spawn(|| {})	threadpool.spawn(|| {}) (assume 8 threads)	tokio::spawn(async {}) (or manual event loop – anything with non-blocking IO)
A million machine learning tasks with some dependencies	✅ or ❌?		
Parallel prefix sums			
Server with millions of simultaneous client connections			

For each task → Use case ↓	thread::spawn(|| {})	threadpool.spawn(|| {}) (assume 8 threads)	tokio::spawn(async {}) (or manual event loop – anything with non-blocking IO)
A million machine learning tasks with some dependencies	❌ Out of memory.	✅ Run no more than 8 things in parallel	❌ No advantage for parallelism and CPU-bound work.
Parallel prefix sums			
Server with millions of simultaneous client connections			

For each task → Use case ↓	thread::spawn(|| {})	threadpool.spawn(|| {}) (assume 8 threads)	tokio::spawn(async {}) (or manual event loop – anything with non-blocking IO)
A million machine learning tasks with some dependencies	❌ Out of memory.	✅ Run no more than 8 things in parallel	❌ No advantage for parallelism and CPU-bound work.
Parallel prefix sums	✅ If you spawn around 8 threads.	✅ Similar to spawning 8 threads	❌ No advantage for parallelism and CPU-bound work.
Server with millions of simultaneous client connections			

For each task → Use case ↓	thread::spawn(|| {})	threadpool.spawn(|| {}) (assume 8 threads)	tokio::spawn(async {}) (or manual event loop – anything with non-blocking IO)
A million machine learning tasks with some dependencies	❌ Out of memory.	✅ Run no more than 8 things in parallel	❌ No advantage for parallelism and CPU-bound work.
Parallel prefix sums	✅ If you spawn around 8 threads.	✅ Similar to spawning 8 threads	❌ No advantage for parallelism and CPU-bound work.
Server with millions of simultaneous client connections	❌ Out of memory.	❌ Prevents more than 8 client connections at a time.	✅ Tasks spend most of their time waiting on IO. Low memory usage.